Chapitre 0 — Avant tout | Partie A — Écrire des lettres | leçon 0.0.1 sur 153 | difficulté 0 / 10

0.0.1. Pourquoi écrire « #include » ?

Une fonction de la console prend de la place dans la cartouche. On écrit « #include <nom> » pour dire qu'on s'en sert : sans la ligne, le compilateur la refuse ; avec, il la grave — et seulement si le programme l'emploie vraiment.

1. Ce qu'il faut comprendre

Au 0.0, le programme ne faisait rien, et il n'avait aucune ligne `#include`. À partir de maintenant, presque chaque programme commence par quelques lignes `#include <…>`. Cette étape explique pourquoi, une fois pour toutes, avant d'en écrire une.

La cartouche est petite. Une cartouche Game Boy, c'est **32 Ko** : 32 768 octets pour tout le jeu — le programme, les dessins, la musique. Un octet est une case qui retient un nombre de 0 à 255. Chaque instruction du programme en occupe quelques-unes.

Une fonction de la console prend de la place. Écrire un mot (`texte()`), lire la manette (`bouton()`), jouer une note (`note()`) : derrière chacune, il y a du code, des petites routines, des tables (les dessins des lettres, les hauteurs des notes). Tout cela doit être **gravé dans la cartouche** pour que la console puisse s'en servir.

Rien n'est là d'office. Si la console mettait toutes ses fonctions dans chaque cartouche, un programme qui n'écrit qu'une lettre emporterait aussi la musique, les lutins, la police entière… des milliers d'octets pour rien. La règle est donc celle du langage C : **ce qu'on veut employer, on l'inclut**.

La ligne `#include <texte>` se lit : « ce programme se sert de `texte()` ». Le **nom entre les chevrons `< >`** est celui de la fonction, écrit **exactement** comme dans le programme : `#include <texte>` pour `texte()`, `#include <ALPHABET>` pour `ALPHABET`, `#include <Tuile>` pour une `Tuile`. Le `#` au début dit que ce n'est pas une instruction du jeu : c'est une indication donnée au compilateur, avant la compilation.

Sans la ligne, le compilateur refuse, et il dit laquelle écrire : « il faut « #include <texte> » pour employer texte() ». Personne n'a à deviner. Dans l'atelier, un bouton l'écrit pour toi.

Avec la ligne, il ne grave que ce qui sert. Une ligne `#include` de trop ne coûte **rien** : si le programme n'appelle jamais la fonction, rien n'est gravé. Le panneau ROM de l'atelier dit, pour chaque ligne, si elle sert et combien d'octets elle coûte.

Ce qui ne s'inclut pas : `image()`, `images()`, `retard()`, `ms()` et `secondes()` sont **natives** — elles font tourner la boucle du jeu et ne coûtent rien. C'est pourquoi le 0.0 n'avait besoin d'aucun `#include`.

La suite du parcours avance une ligne à la fois. Chaque fois qu'une fonction de la console sert pour la première fois, une étape la présente seule — sa ligne `#include`, ses arguments, ce qu'elle coûte —, juste avant la leçon qui l'emploie. Tu n'as jamais deux lignes nouvelles à comprendre en même temps.

Essaie : efface la ligne `#include <texte>`, puis lance : lis le message du compilateur. Remets-la : tout repart.

> **Les fonctions de cette leçon — et le tuto de chacune**
>
> - `#include <texte>` → 0.0.2. La fonction texte() — écrire un mot

📖 Les mots de cette leçon — si tu débutes

- **accolades { }** — Elles entourent un bloc : les lignes qui vont ensemble (le corps d'une fonction, d'une boucle, d'un « if »). Chaque « { » a son « } ».
- **argument** — Une valeur donnée à une fonction quand on l'appelle, entre ses parenthèses : dans texte(5, 8, "SALUT"), 5, 8 et "SALUT" sont les arguments.
- **bloc** — Un groupe de lignes entre accolades { }, qui s'exécutent ensemble.
- **booléen (vrai / faux)** — Le résultat d'une condition : vrai (1) ou faux (0). « true » veut dire vrai, « false » faux.
- **boucle** — Des instructions qui se répètent : « while » (tant que…) ou « for » (pour un compteur qui va de… à…).
- **boucle du jeu** — « while (true) { image(); … } » : ce qui se répète 60 fois par seconde, sans fin, pendant tout le jeu.
- **chaîne de caractères (texte)** — Des lettres entre guillemets, "COMME CECI" : un texte pris tel quel. La police de la console n'a que des majuscules, sans accents.
- **commentaire (//)** — Tout ce qui suit « // » jusqu'au bout de la ligne : une phrase pour l'humain, que la console ignore.
- **condition** — Une question dont la réponse est vrai ou faux : « score > 10 », « bouton(A) ». On la teste avec « if » ou « while ».
- **coordonnées** — Deux nombres qui disent où : la colonne (vers la droite) puis la ligne (vers le bas), à partir de (0, 0) en haut à gauche.
- **image()** — Attend l'image suivante. La console en montre 60 par seconde : c'est ce qui règle la vitesse d'un jeu.
- **#include** — En haut du programme : « #include <texte> » demande la fonction texte() à la console ; « #include "fichier.cpp" » verse un de tes fichiers à cet endroit.
- **instruction** — Un ordre donné à la console, terminé par un point-virgule : « texte(1, 1, "OK"); ».
- **main** — « int main() { … } » : là où le programme commence. La console exécute d'abord la première ligne de main.
- **point-virgule (;)** — Il termine une instruction, comme le point termine une phrase.
- **texte()** — Écrit des lettres à l'écran, sur la grille de 20 colonnes et 18 lignes : texte(colonne, ligne, "MOT").
- **while (tant que)** — Répète un bloc tant qu'une condition est vraie.

🔍 Le programme, ligne par ligne — chaque ligne dite en français

`// ---- Pourquoi écrire #include ? ----`	Un commentaire : une phrase pour celui qui lit (« ---- Pourquoi écrire #include ? ---- »). La console l'ignore.
`// La ligne ci-dessous dit au compilateur : « ce programme se sert de texte() ».`	Un commentaire : une phrase pour celui qui lit (« La ligne ci-dessous dit au compilateur : « ce programme se sert de texte() ». »). La console l'ignore.
`//   #include <texte>`	Un commentaire : une phrase pour celui qui lit (« #include <texte> »). La console l'ignore.
`//   |       |`	Un commentaire : une phrase pour celui qui lit (« | | »). La console l'ignore.
`//   |       +-- le nom de la fonction, écrit comme dans le programme`	Un commentaire : une phrase pour celui qui lit (« | +-- le nom de la fonction, écrit comme dans le programme »). La console l'ignore.

`//   +---------- « # » : une indication pour le compilateur, pas une instruction du jeu`	Un commentaire : une phrase pour celui qui lit (« +---------- « # » : une indication pour le compilateur, pas une instruction du jeu »). La console l'ignore.
`// Sans elle, texte() est refusée : une fonction de la console prend de la`	Un commentaire : une phrase pour celui qui lit (« Sans elle, texte() est refusée : une fonction de la console prend de la »). La console l'ignore.
`// place dans la cartouche, et ne s'emploie qu'incluse. Avec elle, texte() est`	Un commentaire : une phrase pour celui qui lit (« place dans la cartouche, et ne s'emploie qu'incluse. Avec elle, texte() est »). La console l'ignore.
`// gravée — et seulement parce que le programme l'appelle vraiment.`	Un commentaire : une phrase pour celui qui lit (« gravée — et seulement parce que le programme l'appelle vraiment. »). La console l'ignore.
`#include <texte>   // écrit un texte à l'écran`	Demande à la console « texte » (ce que c'est : écrit un texte à l'écran, à la taille qu'on veut). Sans cette ligne, le compilateur refuserait de l'employer. (Le commentaire le dit : « écrit un texte à l'écran ».)
`int main() {`	Le début du programme : la console commence ici, par la première ligne qui suit.
`texte(1, 8, "INCLUS DONC GRAVE");   // texte() existe : sa ligne #include est là`	Appel de la fonction texte de la console, avec les arguments 1, 8, "INCLUS DONC GRAVE". Ce qu'elle fait : écrit un texte à l'écran — 20 colonnes, 18 lignes. Un nombre se colle à la suite : "SCORE " + score. Avec un Mot : texte(SALUT). Un 4e argument est une TAILLE, sans limite : 1 normal, 2 deux fois plus grand, 3… — texte(2, 2, "SUPER", 3). Avec une taille, le texte et la taille s'écrivent en clair ; ce qui dépasse de l'écran est coupé. (Le commentaire le dit : « texte() existe : sa ligne #include est là ».)
`while (true) {     // la boucle du jeu`	La boucle du jeu : tout ce qui est entre les accolades recommence sans fin, 60 fois par seconde avec image(). (Le commentaire le dit : « la boucle du jeu ».)
`image();         // image() est native : elle ne s'inclut pas`	Attendre l'image suivante : la console en montre 60 par seconde. C'est ce qui règle la vitesse du jeu. (Le commentaire le dit : « image() est native : elle ne s'inclut pas ».)
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».
`}`	L'accolade fermante : la fin du bloc ouvert plus haut par « { ».

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **474 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

```
Gravé dans la cartouche :
  démarrage            49 o  ← la console, à l'allumage  — la mise en route de la console
  main()               18 o  ← démarrage  — ta fonction
  VBlank                8 o  ← la console, à chaque image  — compte les images, 60 fois par seconde
  AttendreImage        28 o  ← image()  — attend l'image suivante
  AttendreVBlank       16 o  ← démarrage  — attend que l'écran soit libre
  AttendreAcces        12 o  ← EcrireTexte  — attend de pouvoir écrire une case
  EcrireTexte          10 o  ← texte()  — écrit un texte, lettre par lettre
  CopierTuiles         94 o  ← démarrage  — copie les lettres et les dessins en mémoire vidéo
```

```
  EffacerCarte         14 o  ← démarrage  — vide les deux cartes de l'écran
  Tuiles              208 o  ← CopierTuiles  — 13 lettres sur 43
  "INCLUS DONC GRA…"   17 o  ← texte()  — un texte gravé
Lettres gravées : ACDEGILNORSUV
Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, AdresseCase, EffacerCases, EcrireNombre, Diviser, Reste,
Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau,
LireManette, DonneesTransfert, Notes
Fonctions du langage appelées :
  texte() × 1    11 o sur place, dans main()  + EcrireTexte (10 o)
  image() × 1     3 o sur place, dans main()  + AttendreImage (28 o)
```

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- Pourquoi écrire #include ? ----
// La ligne ci-dessous dit au compilateur : « ce programme se sert de texte() ».
//
//   #include <texte>
//   |        |
//   |        +-- le nom de la fonction, écrit comme dans le programme
//   +----------- « # » : une indication pour le compilateur, pas une instruction du jeu
//
// Sans elle, texte() est refusée : une fonction de la console prend de la
// place dans la cartouche, et ne s'emploie qu'incluse. Avec elle, texte() est
// gravée — et seulement parce que le programme l'appelle vraiment.

#include <texte>   // écrit un texte à l'écran
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

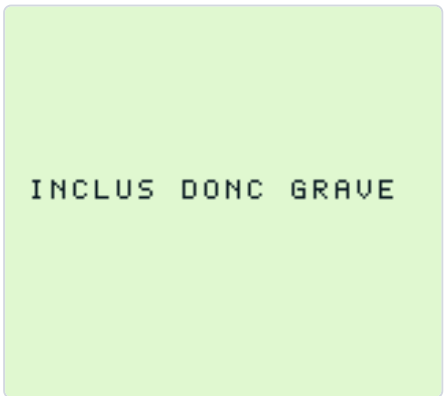
```
int main() {
  texte(1, 8, "INCLUS DONC GRAVE");   // texte() existe : sa ligne #include est là

  while (true) {     // la boucle du jeu
    image();         // image() est native : elle ne s'inclut pas
  }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.

la toute première image — la console vient de démarrer — image 1



un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce qu'on doit voir

> INCLUS DONC GRAVE écrit au milieu de l'écran.

5. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le texte est écrit : texte() est bien incluse

Ensuite : **0.0.2. La fonction texte() — écrire un mot** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++